

Visual Test Oracle: Engineering Case Study

Standalone engineering case study for multimodal GUI regression testing

Repository: <https://github.com/pthanureddy/visual-test-oracle>

Report version: July 2026

Abstract

Graphical user interfaces are difficult to test with conventional automation alone because many important failures are visual, contextual, or tolerant of harmless layout variation. A strict selector assertion can confirm that a button exists, but it cannot judge whether the final screen looks coherent to a user. A pixel comparison can catch visual drift, but it tends to be brittle when fonts, spacing, copy, or rendering engines change. The Visual Test Oracle project explores a practical middle ground: a Playwright-driven test runner captures checkpoint screenshots, then an oracle judges whether the screenshot satisfies a natural-language expected outcome.

The system is built as a reproducible engineering prototype. It includes a small checkout UI, YAML test specifications, controlled UI mutants, baseline assertions, screenshot capture, a deterministic CI-safe oracle, optional OpenAI and Ollama vision adapters, k-run majority voting, and a self-healing locator fallback. The central design question is not whether a model can replace all test assertions. It is whether visual checking can become a measurable, repeatable layer in a regression test suite. This case study documents the system architecture, implementation choices, evaluation method, current results, limitations, and reproduction steps.

The current baseline evaluation includes 14 cases. The deterministic oracle catches 90.00 percent of controlled mutants, produces a 0.00 percent false positive rate on clean runs, preserves 100.00 percent maintenance resilience on benign UI variation, and demonstrates one successful self-healing recovery for a renamed selector. A bounded local-model comparison using qwen2.5vl:latest is included as a separate artifact. The project should be read as an engineering case study: it is intentionally scoped, measurable, and transparent about where deterministic checks end and model-backed judgment begins.

Table of Contents

1. Problem framing
2. Background and related work
3. System goals and non-goals
4. Architecture
5. Test specification design
6. Execution and screenshot capture
7. Oracle design and k-run voting
8. Self-healing locator fallback
9. Controlled mutants and evaluation design
10. Results and interpretation
11. Reproducibility and operations
12. Limitations, risks, and future work
13. References

1. Problem Framing

Modern web interfaces fail in ways that are hard to reduce to a single DOM assertion. A checkout screen may have the correct URL and a visible confirmation element while still presenting a wrong color state, an incorrect total, a broken layout, or a cart count that did not reset. Traditional end-to-end tests are very good at exercising flows, but their strongest assertions are usually exact selectors, exact text, attributes, and element state. Those checks are necessary, yet they are incomplete for user-facing regressions.

The test-oracle problem describes the difficulty of deciding whether observed system behavior is correct. Barr et al. describe oracle automation as a major bottleneck for automated testing because test execution is easier to automate than the judgment of correctness [1]. GUI testing makes that bottleneck more visible. A visible interface has semantic meaning, visual hierarchy, and user expectations that do not always appear in machine-readable DOM fields.

Visual Test Oracle treats a checkpoint screenshot as evidence. The expected outcome is written in ordinary language, for example: a green confirmation banner is visible with text close to "Order placed" and the cart icon shows the number 0. The runner performs the actions, captures the screenshot, gathers a DOM snapshot, and asks an oracle for a pass or fail judgment. The implementation keeps a deterministic fallback so the system can run in CI without secrets or model cost, while the same interface can be routed to hosted or local vision models for experimental runs.

This design does not remove conventional tests. It adds a second layer that can answer questions that selectors and exact text matching answer poorly. The prototype is designed to make that layer observable: every case records the expected verdict, actual verdict, confidence, agreement rate, screenshot path, DOM snapshot, and self-healing events.

Testing concern	Conventional assertion	Visual oracle contribution
Button exists	Strong	Not primary
Final state looks correct	Weak	Strong
Copy is close but not exact	Brittle	Tolerant
Visual color semantics	Possible but verbose	Directly expressed
Benign selector rename	Fails without maintenance	Can recover with fallback

2. Background and Related Work

The project sits at the intersection of test oracles, visual GUI automation, and model-backed judgment. The test oracle survey by Barr et al. gives the core framing: testing requires not only inputs and execution, but also an automated way to decide whether the output is acceptable [1]. For GUI systems, that decision is complicated by visual presentation, accessibility semantics, layout, and user expectations.

Screenshot-driven GUI automation has a long history. Sikuli demonstrated that screenshots can be used as a practical interface for search and automation, allowing tools to reason over visible UI artifacts rather than only source-level or DOM-level structures [2]. That idea is important here because the screenshot is the common artifact seen by both a human reviewer and a multimodal model.

Visual GUI testing research also highlights the tension between robustness and precision. Image-based techniques can be more natural for user-facing checks, but they must manage rendering variance, screen resolution, dynamic content, and maintenance cost. Recent work on LLM-assisted web element localization reports that language models can help select likely target elements when conventional similarity ranking fails, reducing failed localizations in a web-element dataset [3]. Visual Test Oracle uses the same engineering direction in a smaller way: a failed CSS selector can fall back to a target description and a screenshot-guided location request.

The implementation uses Playwright because it provides reliable browser automation, screenshot capture, and test integration. The Playwright documentation exposes screenshot capture directly through the page API [4]. For optional multimodal evaluation, the project uses OpenAI vision-capable APIs and the Responses interface, which support image input and text output [5], [6]. These model-backed paths are optional because a public repository should still run without private credentials.

The engineering position is therefore conservative. The prototype does not claim that vision models are perfect oracles. It uses them as one measurable oracle candidate, surrounds them with consistency voting, and records every decision so that failures are visible.

3. System Goals and Non-goals

The first goal is to make visual oracle behavior reproducible. Every test case is declared in YAML, every mutation is deterministic, and every run writes machine-readable evaluation output. This matters because the main risk in model-backed testing is that impressive demos can hide weak repeatability. A useful test tool must produce artifacts that can be inspected, compared, and regenerated.

The second goal is to separate test intent from test mechanics. YAML specs describe the route, action sequence, target selector, target description, checkpoint, and expected outcome. The test runner owns browser mechanics. The oracle owns final-state judgment. This separation makes it possible to compare baseline assertions, deterministic oracle checks, hosted vision models, and local vision models without rewriting the test intent.

The third goal is to study maintenance resilience. GUI tests often fail when a selector changes even though the user-visible workflow still works. The project includes a benign selector rename scenario where `#checkout - btn` becomes `#place-order - btn`. The baseline selector test is expected to fail, while the oracle path can recover by using the target description and a visual or accessibility fallback.

The fourth goal is to keep public CI clean. GitHub Actions runs unit tests, browser checks, and the deterministic report generation path without requiring `OPENAI_API_KEY`, Ollama, or paid model calls. This makes the repository safe to clone, fork, and evaluate.

The project intentionally has non-goals. It does not implement a full commercial test platform. It does not attempt broad browser/device coverage. It does not claim a general benchmark for all GUI defects. It also does not treat the deterministic fallback as a replacement for a real vision model. The fallback is a reproducibility mechanism and a CI safety mechanism.

Goal	Implementation choice
Reproducible checks	YAML specs, deterministic mutants, JSON artifacts
Visual judgment	Screenshot checkpoint and oracle abstraction
Non-determinism tracking	k-run majority vote and agreement rate
Maintenance resilience	Self-healing locator fallback
Public CI safety	Deterministic oracle path without secrets

4. Architecture

The architecture has five main layers: the static checkout UI, the Playwright runner, the oracle interface, the consistency layer, and the reporting layer. The checkout UI is intentionally small. It contains enough states to model a meaningful flow: cart count, order total, checkout button, confirmation banner, color semantics, and bug-mode toggles. A small UI makes ground truth clear, which is important for controlled evaluation.

The Playwright runner serves the static UI locally, executes YAML-defined steps, captures screenshots, collects DOM evidence, and writes per-case results. Each result includes the case id, mutation, variant, expected verdict, actual verdict, agreement rate, confidence, oracle votes, self-healing events, and screenshot path.

The oracle interface supports multiple providers. The deterministic provider judges the DOM snapshot and is used for CI. The OpenAI provider sends the screenshot plus expected outcome to a vision-capable model and expects a structured JSON verdict. The Ollama provider sends the screenshot to qwen2.5vl:latest through a local endpoint. The provider interface is intentionally narrow: judge returns a verdict, confidence, reasoning, provider name, latency, and cost estimate; locate optionally returns a screen coordinate for self-healing.

The consistency layer runs the oracle multiple times and performs majority vote. It records agreement rate rather than hiding it. A unanimous pass is different from a two-to-one pass, especially when a probabilistic model is involved.

```
YAML spec -> Playwright runner -> screenshot + DOM snapshot
          -> oracle provider -> k-run vote -> JSON artifacts
          -> report generator -> GitHub Pages dashboard + case study
```

This architecture is deliberately simple. Each layer can be tested independently, and each output can be inspected without a special service.

5. Test Specification Design

The test specification format is short enough to review in code review but expressive enough to support visual judgment. Each spec includes a stable id, a description, a sequence of steps, a checkpoint type, and a natural-language expected outcome. Click steps include both a CSS selector and a target description. The selector is used first because selectors are fast and precise. The description is kept as a fallback because it survives benign selector changes better than a hard-coded id.

```
test_id: checkout_success_state
description: Completes the checkout flow and evaluates the final screen.
steps:
  - action: goto
    url: "/"
  - action: click
    selector: "#checkout-btn"
    target_description: "the main checkout or place order button"
  - action: wait
    ms: 300
checkpoint: screenshot
expected_outcome: >
  A green confirmation banner is visible with text close to "Order placed".
  The cart icon shows the number 0.
```

This format makes the expected outcome readable to humans and suitable for model-backed evaluation. It also avoids burying the user-visible requirement inside Python code. A reviewer can understand what the test is trying to prove without reading the runner.

There are three current specs. `checkout_success_state` checks the main success flow. `checkout_total_state` focuses on total visibility and currency. `selector_healing_checkout` exercises maintenance resilience by using a selector that fails under the renamed-selector variant but preserving the target description.

The format can grow without changing the core design. Future fields could add viewport, accessibility expectation, screenshot region, or severity. The current implementation keeps the schema minimal to reduce false precision.

6. Execution and Screenshot Capture

The execution layer uses a local static server and Playwright Chromium. A test case is built from a spec, a bug mode, a benign variant, an expected verdict, and an oracle provider. The runner opens the checkout UI with query-string toggles, performs the declared steps, captures a full-page screenshot, and records a DOM snapshot.

The screenshot is the main visual artifact. The DOM snapshot is supporting evidence. For the deterministic provider, the DOM snapshot provides reproducible checks for confirmation visibility, banner text, semantic color role, cart count, and total. For model-backed providers, the screenshot is the main input and the DOM snapshot is useful for debugging and comparison.

Playwright is a good fit because it supports reliable browser control and direct screenshot capture through the page API [4]. The project uses a full-page screenshot because the UI is compact and because the expected outcome spans multiple visual regions: banner, cart count, and total. In a larger system, the screenshot could be clipped to a region to reduce model cost and improve focus.

The baseline tests remain important. They prove the conventional path works on the clean flow and demonstrate brittleness under a benign selector rename. This comparison is not adversarial. Selector assertions should remain part of a test suite. The point is that they measure a different property from the visual oracle.

Artifact	Purpose
Screenshot PNG	Visual evidence at checkpoint
DOM snapshot	Deterministic evidence and debugging support
Evaluation JSON	Machine-readable case results
GitHub Pages report	Public summary and evidence gallery
Case-study PDF	Formal engineering documentation

7. Oracle Design and K-run Voting

The oracle prompt asks for a structured JSON response with `verdict`, `confidence`, and `reasoning`. This is intentionally strict. Free-form answers are hard to aggregate and easy to misread. The parser rejects invalid verdicts so malformed model output becomes visible rather than silently passing.

The deterministic oracle exists for two reasons. First, it makes CI safe. Second, it provides a stable baseline for evaluating the rest of the pipeline. It checks the DOM evidence against the expected outcome and returns a verdict in the same shape as a model-backed provider. That keeps the reporting code independent of the provider.

For model-backed runs, the OpenAI provider uses a vision-capable endpoint with a screenshot and natural-language expected outcome. OpenAI documents vision input for image understanding and the Responses interface for text and image input with text output [5], [6]. The Ollama provider uses a local `qwen2.5vl:latest` model through the local generate endpoint. This local path gives an engineering comparison point for latency, cost, and repeatability.

The k-run voting layer is the guardrail around non-determinism. The default `k=3` path runs the same oracle three times and takes the majority verdict. The agreement rate is logged. This creates a simple but useful signal:

Votes	Final verdict	Agreement
pass, pass, pass	pass	1.00
pass, pass, fail	pass	0.67
fail, fail, pass	fail	0.67

The project does not treat confidence as a magic truth score. Confidence is recorded and summarized, but majority verdict and agreement are more transparent for regression use.

8. Self-healing Locator Fallback

Self-healing begins only after a normal selector click fails. The runner first tries the CSS selector from the YAML spec. If that click times out, it captures a pre-healing screenshot and asks the oracle to locate the element described by `target_description`. If the provider cannot return a coordinate, the runner falls back to an accessibility/text query for visible buttons such as "Checkout" or "Place order".

This design keeps the fast path simple. It does not replace selectors with model calls. Instead, it uses the model or fallback only when the selector path breaks. That matters because model calls are slower, can cost money, and can be less predictable than direct selectors. The self-healing path is a maintenance fallback, not the primary interaction mechanism.

The renamed-selector scenario demonstrates the value. The visual UI is still valid, but `#checkout-btn` no longer exists. The baseline selector test fails as expected. The oracle runner records a healing event and completes the flow. The final verdict is pass because the user-visible outcome is still correct.

The current evaluation distinguishes healing attempts from healing successes. A hidden button or disabled button may trigger the healing code, but that is not a success because the flow should not pass. The reported `self_healing_successes` metric counts recovered passing flows only.

Risks remain. Coordinate-based clicking can hit the wrong target if the model returns an imprecise box. Accessibility fallback can be too broad if many buttons share similar labels. For production use, a healed click should be verified by checking that the intended state transition occurred. The prototype records enough evidence to support that next step.

9. Controlled Mutants and Evaluation Design

The checkout UI includes deterministic bug modes. Each mode changes one user-visible property. This makes expected verdicts clear and avoids relying on random defects. The mutants cover hidden controls, disabled controls, wrong color semantics, wrong copy, cart count failure, missing total, wrong currency, layout shift, missing confirmation, and an extra error state.

Controlled mutants let the project compute bug detection rate. Clean runs compute false positive rate. Benign variants compute maintenance resilience. The renamed-selector variant computes self-healing recovery. This evaluation design is small, but it is explicit and reproducible.

Case family	Expected result	Metric contribution
Clean checkout	pass	False positive baseline
Controlled bug mode	fail	Bug detection rate
Benign copy change	pass	Maintenance resilience
Renamed selector	pass after recovery	Self-healing success
Ollama bounded subset	mixed	Local-model comparison

One limitation is that not all visual defects are equally represented. A layout shift can be subjective: the current deterministic oracle does not fail a layout shift unless it changes a checked semantic property. A real vision model might judge a severe layout shift differently. This is why the project reports the layout-shift case explicitly rather than hiding it inside an aggregate metric.

The evaluation output is stored in `artifacts/evaluation.json`. The bounded local-model comparison is stored in `artifacts/ollama_bounded_evaluation.json`. Keeping the artifacts in the repository makes the current result auditable without rerunning model calls.

10. Results and Interpretation

The current deterministic evaluation produced 14 cases. The main metrics are:

Metric	Value
Total cases	14
Bug detection rate	90.00%
False positive rate	0.00%
Maintenance resilience	100.00%
Mean agreement rate	100.00%
Self-healing successes	1

The false positive result is important. A visual oracle that constantly fails clean screens is not useful, even if it catches bugs. The deterministic baseline produces no false positives on the clean checkout cases. The maintenance resilience result is also important: the benign copy change remains a pass.

The missed mutant is the layout-shift case. This is not a hidden failure in the report; it is an example of the boundary between semantic and visual judgment. The deterministic oracle checks the confirmation state, cart state, and total state. It does not currently encode layout aesthetics. A model-backed provider might catch this case, but it would also need repeatability analysis. The correct engineering response is to add a visual layout expectation or region-specific check rather than overclaiming the baseline.

The bounded Ollama run confirms that the local-provider path works, but it should not be treated as a full benchmark. It was run with $k=1$ for practical runtime reasons and produced a much higher mean latency. This is useful operational evidence: local models can remove hosted API cost, but the hardware/runtime tradeoff is real.

The result supports a narrow conclusion: a screenshot-oracle layer can be integrated into an automated GUI test suite in a reproducible way, and it can expose useful signals beyond exact selector assertions.

11. Reproducibility and Operations

The public CI path uses the deterministic oracle and does not require secrets. A local developer can reproduce the baseline with:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m playwright install chromium
pytest
python scripts/run_evaluation.py --provider heuristic --k 3
python scripts/build_case_study_pdf.py
```

The hosted model path requires an OpenAI key:

```
$env:OPENAI_API_KEY="<your-openai-api-key>"
$env:OPENAI_MODEL="gpt-5.5"
python scripts/run_evaluation.py --provider openai --k 3
```

The local model path requires Ollama and the vision model:

```
ollama pull qwen2.5vl:latest
python scripts/run_evaluation.py --provider heuristic --include-ollama --k 1
```

Operationally, model-backed test runs should be separated from pull-request CI unless the team is prepared to manage cost, latency, and flakiness. One practical pattern is to run deterministic checks on every commit, run model-backed checks on a schedule, and require human review when model agreement is low. The important point is to keep screenshots, prompts, raw votes, and final verdicts as durable artifacts.

12. Limitations, Risks, and Future Work

The prototype is intentionally small. Its checkout UI is controlled, static, and deterministic. That makes it good for method validation but not representative of all production complexity. Dynamic data, animations, third-party widgets, authentication flows, and responsive breakpoints would add more failure modes.

The deterministic oracle is not a vision model. It proves the pipeline and provides stable CI behavior, but it cannot judge broad visual quality. Conversely, a model-backed oracle can judge visual quality but introduces non-determinism, latency, cost, and provider-specific behavior. The k-run voting layer is a mitigation, not a proof of correctness.

The self-healing locator strategy is useful but risky. A healed click can hide a real selector maintenance problem if teams do not review healing events. The report should therefore treat healing as a warning-level recovery, not a silent success. A mature system would open a maintenance issue when healing is used repeatedly.

Future work should add viewport matrix testing, visual regions, accessibility-state checks, stronger prompt versioning, cost tracking, and a larger mutant set. It should also compare at least two hosted vision models and two local models on the same artifacts. A stronger evaluation would include human labels for visual defects and measure agreement among human reviewers, deterministic checks, and model-backed checks.

Despite these limitations, the project demonstrates a clear engineering path. Visual judgment can be added to GUI testing without abandoning conventional assertions. The useful pattern is layered: selectors for precise interaction, screenshots for visual evidence, deterministic checks for CI, model-backed checks for richer judgment, and artifacts for audit.

13. References

- [1] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The Oracle Problem in Software Testing: A Survey," IEEE Transactions on Software Engineering, 2015. <https://ieeexplore.ieee.org/document/6963470>
- [2] T. Yeh, T.-H. Chang, and R. C. Miller, "Sikuli: Using GUI Screenshots for Search and Automation," UIST, 2009. <https://dl.acm.org/doi/10.1145/1622176.1622213>
- [3] M. Nass et al., "Improving Web Element Localization by Using a Large Language Model," Software Testing, Verification and Reliability, 2024. <https://arxiv.org/abs/2310.02046>
- [4] Microsoft Playwright, "Screenshots - Playwright Python Documentation." <https://playwright.dev/python/docs/screenshots>
- [5] OpenAI, "Images and Vision - OpenAI API Documentation." <https://developers.openai.com/api/docs/guides/images-vision>
- [6] OpenAI, "Responses Overview - OpenAI API Reference." <https://developers.openai.com/api/reference/responses/overview/>